Python Engineering - Concept Guide

Class 09 - Modules & Imports: main.py, process_trajectory() y __name__

Guia paso a paso para terminar EX17. El objetivo no es cambiar la fisica, sino separar responsabilidades: cada modulo calcula su parte y main.py coordina el programa y presenta los resultados.

1. Arquitectura mental: tres archivos, tres responsabilidades

Archivo	Responsabilidad	Que NO deberia hacer
vertical_motion.py	Fisica, Euler, simulacion, solucion analitica, errores	Configurar el caso o presentar el informe final
data_processing.py	Mascaras, filtros, estadistica, energia y dataset procesado	Ejecutar la simulacion o decidir la configuracion
main.py	Configurar, llamar funciones y mostrar resultados	Contener ecuaciones de Euler o detalles de filtrado

Regla simple: vertical_motion calcula el vuelo; data_processing analiza los datos; main decide que ejecutar y que mostrar.

2. El flujo completo

```
main.py
  |-- config
  |-- validate_inputs(config)
  |-- simulate_vertical_motion(config)
  |     `--> results, trajectory, trajectory_array
  |-- analytical_solution(config)
  |-- calculate_errors(...)
  |-- process_trajectory(trajectory_array, config)
  |     `--> processing_results, processed_data
  `-- print everything
```

3. Que es process_trajectory()?

Es una funcion que recibe los datos producidos por la simulacion y los convierte en informacion util. No vuelve a calcular la trayectoria: solo la procesa.

Elemento	Contenido
Input 1	trajectory_array: matriz [h, v, t]
Input 2	config: diccionario con g, dt, etc.
Output 1	processing_results: diccionario con resumen, filtros, indices y estadisticas
Output 2	processed_data: matriz [h, v, t, energy]

4. data_processing.py - esqueleto recomendado

```
import numpy as np

def process_trajectory(trajectory_array, config):
    heights = trajectory_array[:, 0]
    velocities = trajectory_array[:, 1]
    times = trajectory_array[:, 2]

    ascending_mask = velocities > 0
    descending_mask = velocities < 0
    above_100_mask = heights > 100
    time_window_mask = (times >= 2.0) & (times <= 5.0)
    high_ascending_mask = (heights > 50) & (velocities > 0)

    energy = config['g'] * heights + 0.5 * velocities**2
    processed_data = np.column_stack((trajectory_array, energy))
    dt_samples = np.diff(times)

    processing_results = {
        'n_ascending': np.sum(ascending_mask),
        'n_descending': np.sum(descending_mask),
        'states_above_100': trajectory_array[above_100_mask],
        'states_time_2_5': trajectory_array[time_window_mask],
        'states_high_ascending': trajectory_array[high_ascending_mask],
        'mean_height': np.mean(heights),
        'std_height': np.std(heights),
        'mean_velocity': np.mean(velocities),
        'std_velocity': np.std(velocities),
        'indices_above_100': np.where(above_100_mask)[0],
        'dt_samples': dt_samples,
```

```
        'dt_constant': np.allclose(dt_samples, config['dt'])
    }
    return processing_results, processed_data
```

La funcion no hace print. Su trabajo termina cuando devuelve datos.

5. Por que devolver un diccionario?

Porque tendremos muchos resultados. Un diccionario permite acceder por nombre en vez de recordar posiciones.

```
processing_results['mean_height']
processing_results['dt_constant']
processing_results['states_above_100']
```

6. Que queda en main.py?

main.py debe parecerse a un director de orquesta: no toca todos los instrumentos; indica cuando toca cada uno.

```
from vertical_motion import (
    validate_inputs, simulate_vertical_motion,
    analytical_solution, calculate_errors
)
from data_processing import process_trajectory

def main():
    config = {'h0':0.0, 'v0':50.0, 't0':0.0, 'g':9.81, 'dt':0.1}

    if validate_inputs(config):
        results, trajectory, trajectory_array = \
            simulate_vertical_motion(config)
        analytical_results = analytical_solution(config)
        differences, errors = calculate_errors(
            analytical_results['analytical_max_altitude'],
            results['max_altitude'],
            analytical_results['analytical_final_velocity'],
            results['final_velocity'],
            analytical_results['analytical_time_of_flight'],
            results['time_of_flight'])
        processing_results, processed_data = \
            process_trajectory(trajectory_array, config)

        # Presentation only from here
        print(results)
        print(analytical_results)
        print(errors)
        print(processing_results)

if __name__ == '__main__':
    main()
```

7. Entonces, si: los prints van en main

Tu intuicion es correcta. En una arquitectura limpia, las funciones calculan y devuelven. main.py decide como presentar esa informacion.

Para EX17 no hace falta eliminar todos los print internos de simulate_vertical_motion() inmediatamente. Pero la direccion profesional es: **calculo separado de presentacion**.

8. __name__ explicado de forma simple

Python crea automaticamente una variable especial llamada `__name__` dentro de cada archivo.

Caso A - ejecutas el archivo directamente:

```
python main.py

# Dentro de main.py:
__name__ == '__main__'
```

Caso B - importas el archivo:

```
import vertical_motion

# Dentro de vertical_motion.py:
__name__ == 'vertical_motion'
```

La condicion pregunta literalmente: **"Soy el archivo que el usuario ha ejecutado directamente?"**

9. Que hace exactamente el main guard?

```
if __name__ == '__main__':
    main()
```

Si ejecutas main.py directamente, la condicion es True y main() se ejecuta. Si otro archivo hace import main, la condicion es False: Python carga las definiciones, pero no lanza automaticamente la simulacion.

Analogia: un archivo puede ser una maquina y una caja de herramientas. Si lo encienden directamente, arranca la maquina; si lo importan, deja disponibles las herramientas.

10. Por que definir def main()?

No es obligatorio, pero organiza el flujo principal dentro de una funcion, evita ejecucion descontrolada al importar y facilita pruebas/reutilizacion.

11. Imports: tres formas

Sintaxis	Que busca	Que deja disponible	Ejemplo
import module	un modulo	el modulo	vertical_motion.simulate_vertical_motion(...)
import module as alias	un modulo	un alias	np.array(...)
from module import func	un nombre concreto	la funcion	simulate_vertical_motion(...)

12. Cada modulo importa lo que usa

Los imports no se comparten magicamente entre archivos. Si data_processing.py usa NumPy, debe contener `import numpy as np`.

```
# data_processing.py
import numpy as np

# vertical_motion.py
from math import sqrt
import numpy as np

# main.py
# No sqrt or np if main does not use them directly.
```

Cuando termines la redistribucion puedes eliminar de main.py imports externos que ya no utilice.

13. Que NO moveria todavia

Mantendria analytical_solution() y calculate_errors() en vertical_motion.py. Forman parte de la validacion numerica/fisica del simulador. Separarlos ahora fragmentaria demasiado el ejercicio.

14. Checklist para terminar EX17

```
[ ] Create data_processing.py
[ ] Add import numpy as np inside it
[ ] Create process_trajectory(trajectory_array, config)
[ ] Move masks/statistics/energy/dt processing into it
[ ] Return processing_results, processed_data
[ ] Import process_trajectory in main.py
[ ] Call analytical_solution() and calculate_errors() from main.py
[ ] Call process_trajectory() from main.py
[ ] Leave final presentation/prints in main.py
[ ] Remove unused imports from main.py
[ ] Run python main.py
[ ] Verify results match EX16
```

15. Arquitectura final de EX17

```
ex17_vertical_motion_modules/
|-- main.py
|   config / orchestration / prints
|-- vertical_motion.py
|   validation / Euler / simulation / analytical / errors
`-- data_processing.py
    masks / filtering / statistics / energy / processed dataset
```